SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO2 ASSESSMENT TOOL 2 – Architecture Design Assignment

Course Code:	CSA10 / CSA1063	**Course Title:**	Software Engineering
CO Assessed:	CO2	**Assessment:**	Assessment Tool 2 – Architecture Design Assignment
Weightage:	20%	**Total Marks:**	25
CO2 Statement:	*Perform detailed requirements analysis, design scalable architectures, and prioritize features with a focus on cross-disciplinary skills*		
Student Name:	N. Lahari	**Reg. No.:**	192473018
Date:	23-10-2026	**Duration:**	60 minutes

Code of Conduct

I N. Lahari (192473018) _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: N. Lahari _____

Annexure A – Architecture Design Assignment

Instructions to Students:

Each student will be assigned an **individual software project problem statement**. Based on the assigned problem, analyze the system requirements and design an appropriate software architecture by applying software engineering principles. Your submission should demonstrate your ability to select, justify, and evaluate a suitable architectural style.

Question

Based on your **assigned problem statement**, perform an **Architecture Design Analysis** and prepare an architecture design report by answering the following questions:

1. Analyze System Requirements

- Study the given problem statement and identify the system objectives.
- Analyze the functional and non-functional requirements that influence the software architecture.
- Identify key quality attributes such as scalability, maintainability, reliability, availability, performance, and security.

2. Select a Suitable Software Architecture

- Select an appropriate architectural style for the proposed system (e.g., Layered Architecture, Client-Server, MVC, Microservices, Event-Driven, Service-Oriented Architecture, Serverless, or Hybrid Architecture).
- Justify why the selected architecture is the most suitable for the given problem.

3. Draw the Software Architecture Diagram

- Design a clear architecture diagram illustrating the overall structure of the proposed system.
- Include all major layers/components, databases, external systems, APIs, and communication mechanisms.
- Use appropriate software architecture notations and labels.

4. Explain Components and Their Interactions

- Describe the purpose and responsibilities of each architectural component.

- Explain how the components communicate and exchange data.
- Illustrate the overall workflow of the system.

5. Discuss Quality Attributes

Evaluate how the proposed architecture addresses the following aspects:

- Scalability
- Security
- Performance
- Reliability
- Maintainability
- Availability

6. Justify Architectural Decisions

- Explain the rationale behind your architectural choices.
- Discuss the trade-offs considered while selecting the architecture.
- Explain how the chosen architecture supports future enhancements, system integration, and long-term maintainability.

1. Analyze System Requirements

1.1 System Objectives

The main objective is to develop an AI-Assisted Customer Support Platform that automatically handles routine customer requests while identifying conversations that require human intervention. The system aims to improve response speed, reduce workload, maintain security, and ensure responsible AI usage.

The objectives are:

- Automatically handle routine customer queries.
- Detect uncertain, ambiguous, sensitive, and high-risk requests.
- Escalate complex cases to human support staff.
- Maintain conversation context and secure customer records.
- Integrate secure APIs and scalable services.
- Monitor system performance and maintain audit records.

1.2 Functional Requirements

1. Customer Interaction: The system should accept customer queries through a user interface and provide appropriate responses.
2. Request Classification: The AI system should classify requests into categories such as general queries, complaints, technical issues, and sensitive requests.
3. Conversation Context Management: The platform should maintain previous conversation details to provide relevant responses.
4. Uncertainty Detection: The system should identify unclear questions, low-confidence AI responses, and high-risk conversations.
5. Human Escalation: The platform should transfer sensitive or complex cases to human support staff.
6. Secure API Integration: The system should communicate securely with external services using APIs.
7. Audit and Monitoring: The platform should maintain interaction logs, escalation records, and system performance information.

1.3 Non-Functional Requirements

1. Scalability: The system should support an increasing number of customers and requests.
2. Security: Customer information and conversation records must be protected through authentication, authorization, and encryption.

3. Performance: The platform should provide quick responses to routine customer queries.
4. Reliability: The system should operate consistently and handle errors without losing important conversation data.
5. Maintainability: Individual components should be easy to update, test, and maintain.
6. Availability: The platform should remain accessible with minimal downtime.
7. Privacy and Fairness: Customer information should be handled responsibly, and AI responses should be monitored for potential bias.



2. Select a Suitable Software Architecture

Selected Architecture: Hybrid Architecture (Layered Architecture with Microservices)

A Hybrid Architecture is suitable for the AI-Assisted Customer Support Platform because it combines the organized structure of layered architecture with the flexibility of microservices.

The system can be divided into the following layers:

1. Presentation Layer: Provides the customer chat interface and the human support staff interface.
2. API and Application Layer: Receives requests, validates inputs, manages authentication, and routes requests to appropriate services.
3. AI Processing Layer: Classifies customer requests, analyzes conversation context, generates responses, and detects uncertainty.

4. Human Escalation Layer: Transfers sensitive, complex, or uncertain conversations to human support staff.
5. Data Layer: Stores customer interaction history, conversation context, escalation details, and audit records.
6. Monitoring Layer: Tracks response time, system errors, service health, and AI performance.

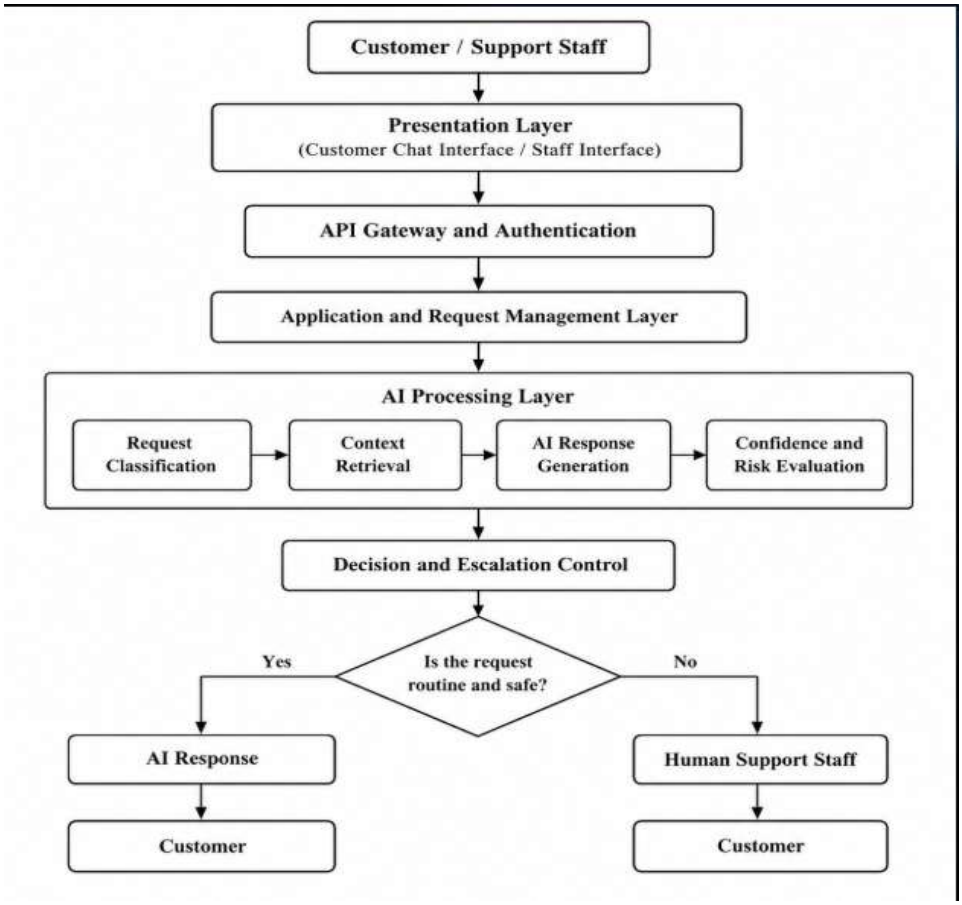
Justification

The hybrid architecture is appropriate because the system contains multiple independent functions, including AI processing, customer interaction, security, and human escalation. Microservices allow individual services to be developed, tested, and scaled independently, while layered architecture separates responsibilities and improves organization.

This approach supports secure communication, easier maintenance, future integration, and reliable customer support. It also ensures that AI automation does not completely replace human judgment in sensitive situations.

3. Software Architecture Diagram

The overall architecture of the proposed system is represented below.



Data Layer:

- Customer Interaction Database
- Conversation Context Storage
- Audit and Escalation Logs

External Services:

- Secure APIs
- AI Model Service
- Notification Service

Monitoring and Logging:

- Response Time
- Error Tracking
- Service Health
- AI Performance

4. Explain Components and Their Interactions

4.1 Customer Interface

The customer interface allows users to submit questions, receive AI-generated responses, and view updates about their support requests. A separate staff interface enables human agents to review escalated conversations and respond to customers.

4.2 API Gateway and Authentication

The API gateway receives incoming requests and forwards them to the appropriate application services. Authentication verifies user identity, while authorization ensures that customers and support staff can access only the information permitted for their roles.

4.3 Request Classification Service

This component analyzes incoming customer messages and classifies them according to their purpose. It identifies routine questions, complaints, technical issues, and requests that may require special attention.

4.4 Conversation Context Service

This service maintains relevant information from previous messages. It retrieves conversation history when needed so that the AI assistant can understand the customer's situation and provide consistent responses.

4.5 AI Response Generation Service

The AI service processes customer queries and generates suitable responses using the available context. It handles routine questions automatically and sends its response for confidence and risk evaluation before delivery.

4.6 Uncertainty and Risk Detection Service

This component evaluates whether an AI-generated response is reliable and appropriate. It identifies ambiguous questions, low-confidence predictions, privacy-sensitive requests, and high-impact situations that require human review.

4.7 Human Escalation Service

When a request is classified as uncertain, sensitive, or high-risk, this service transfers the conversation to a human support agent. It provides the relevant conversation history and reason for escalation so the agent can make an informed decision.

4.8 Database and Audit Service

The database stores customer interactions, conversation context, and support records. The audit service records important events, including AI responses, escalation decisions, and human interventions, to support accountability and troubleshooting.

4.9 Monitoring Service

The monitoring service tracks response time, errors, availability, escalation frequency, and AI performance. These measurements help administrators identify problems and improve the platform.

Overall Workflow

1. The customer submits a query through the chat interface.
2. The API gateway validates and routes the request.
3. The classification service identifies the type of request.
4. The context service retrieves relevant conversation history.
5. The AI service generates a suitable response.
6. The risk detection service evaluates confidence and sensitivity.
7. If the response is safe and reliable, it is sent to the customer.
8. If the request requires intervention, it is escalated to a human support agent.
9. The interaction and decision are recorded in the database and audit logs.
10. The monitoring service tracks system performance and service quality.

5. Discuss Quality Attributes

5.1 Scalability

The architecture supports scalability by allowing individual microservices to scale independently. For example, the AI processing service can be scaled when customer queries increase, without scaling every other component.

5.2 Security

Security is maintained through authentication, role-based authorization, encrypted communication, secure API access, and controlled access to customer records. Sensitive information should be protected during storage and processing.

5.3 Performance

Routine customer requests are processed automatically, reducing the need for manual intervention. Efficient request routing, context retrieval, and independent services help improve response time.

5.4 Reliability

The platform can improve reliability through error handling, service health checks, controlled retries, and recovery mechanisms. Human escalation provides an alternative when the AI cannot produce a reliable response.

5.5 Maintainability

The modular architecture allows developers to update the classification service, AI model, database, or escalation mechanism independently. This reduces the impact of changes and simplifies testing and debugging.

5.6 Availability

The system can improve availability through redundant service instances, health monitoring, recovery procedures, and fault isolation. These measures help maintain customer support when an individual service fails.

6. Justify Architectural Decisions

6.1 Rationale Behind Architectural Choices

A hybrid architecture is selected because it provides modularity, scalability, flexibility, and separation of responsibilities. Layered architecture organizes the system into logical levels, while microservices allow important functions to operate independently.

Human escalation control is included to prevent inappropriate reliance on AI-generated responses. Sensitive,

uncertain, or high-risk conversations are transferred to human staff, improving accountability and customer trust.

6.2 Architectural Trade-Offs

- Advantages: Independent scaling, modular development, improved maintainability, flexible integration, and better separation of responsibilities.
- Challenges: Microservices introduce additional communication overhead, deployment complexity, and monitoring requirements.
- Security Considerations: Multiple services require consistent authentication, authorization, and secure communication.
- Performance Considerations: AI processing and database access may increase response time, so efficient routing and caching may be required.

These trade-offs can be managed through centralized monitoring, secure API communication, automated testing, and appropriate service design.

6.3 Future Enhancements

The platform can be enhanced by integrating multilingual AI support, advanced sentiment analysis, improved uncertainty detection, and additional customer communication channels. It can also use feedback from human agents to improve response quality.

Further enhancements may include automated testing, CI/CD pipelines, container orchestration, advanced analytics, and improved monitoring. These improvements can help the platform support more customers while maintaining security, reliability, and responsible AI practices.

Conclusion

The proposed AI-Assisted Customer Support Platform combines AI automation with human escalation control to provide efficient and responsible customer service. The hybrid architecture supports modular development, scalability, security, and maintainability. By identifying uncertain and sensitive conversations and transferring them to human agents, the system reduces operational risks while improving response speed, service reliability, and customer trust.

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
System Requirement Analysis (30%)	Thoroughly analyzes the problem statement, accurately identifies functional and non-functional requirements, and clearly explains quality attributes influencing the architecture.	Analyzes most requirements with minor omissions or limited explanation.	Identifies only basic requirements with partial analysis.	Requirement analysis is incomplete, inaccurate, or lacks clarity.
Selection of Software Architecture (25%)	Selects the most appropriate architectural style with strong technical justification aligned to system requirements.	Selects a suitable architecture with reasonable justification.	Architecture is partially suitable with limited justification.	Inappropriate architecture selected or justification is missing.
Architecture Diagram and Component Design (20%)	Produces a clear, complete, and well-structured architecture diagram with all major components, interfaces, and interactions accurately represented.	Diagram is mostly complete with minor omissions or notation issues.	Diagram includes only essential components and lacks sufficient detail.	Diagram is incomplete, unclear, or technically incorrect.
Component Interaction and Quality Attribute Analysis (15%)	Clearly explains component responsibilities, interactions, and thoroughly discusses scalability, security, performance, reliability, maintainability, and availability.	Explains most components and quality attributes with minor gaps.	Provides limited explanation of interactions and addresses only some quality attributes.	Component interactions and quality attributes are poorly explained or missing.
Architectural Justification and Technical Presentation (10%)	Provides strong justification for architectural decisions, discusses trade-offs and future enhancements, and presents a well-organized, professional report.	Provides adequate justification with minor gaps; report is well organized.	Limited justification with minimal discussion of trade-offs or future scope; report needs improvement.	Justification is weak or absent; report lacks organization and technical clarity.

Criteria	**Mark**
System Requirement Analysis **(30%)**	/5
Selection of Software Architecture **(25%)**	/5
Architecture Diagram and Component Design **(20%)**	/5
Component Interaction and Quality Attribute Analysis **(15%)**	/5
Architectural Justification and Technical Presentation **(10%)**	/5
TOTAL	/25